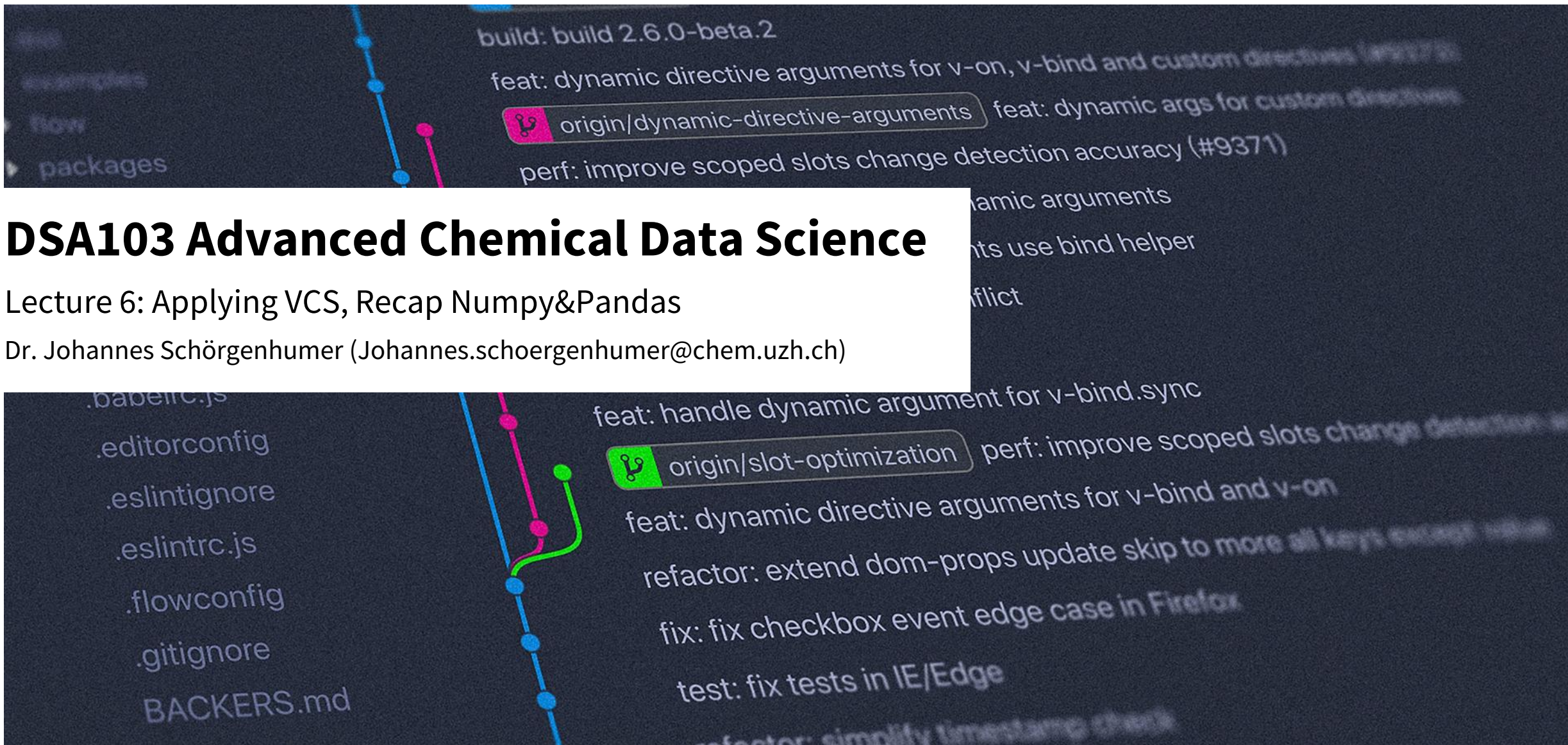




DSA103 Advanced Chemical Data Science

Lecture 6: Applying VCS, Recap Numpy&Pandas

Dr. Johannes Schörgenhumer (Johannes.schoergenheimer@chem.uzh.ch)



Content overview

Discussion Assignment

Recap: Numpy (Arrays, slicing, universal functions, aggregations)

Activity: Numpy

Recap: Pandas (Series, DataFrames, import data; aggregate, transform, filter data; Grouping)

Activity: Pandas

Assignment

Discussion of homework

Exercise 1: Chemical Weight Calculator

Objective: Practice variable assignment, types, and basic data structures while calculating molecular properties.

Problem: Create a molecular weight calculator.

Your Task: Write functions that work with chemical formulas stored in different data structures.

```
# Given data structures
atomic_masses = {
    "H": 1.008, "C": 12.01, "N": 14.007, "O": 15.999,
    "Na": 22.990, "Cl": 35.45, "Ca": 40.078, "S": 32.06
}

# Different ways to represent water (H2O)
water_dict = {"H": 2, "O": 1}
water_list = ["H", "H", "O"]
water_tuple = (("H", 2), ("O", 1))
```

Calculate mass from:

- Dict
- List
- Tuple

How did you solve it?

Chemical Weight Calculator

```
from typing import Dict

def calculate_mass_from_dict(formula_dict: Dict[str, int], masses: Dict[str, float]):
    """Calculate molecular mass from dictionary format"""
    return sum(masses[atom]*amount for atom, amount in formula_dict.items())

calculate_mass_from_dict({"H": 2, "O": 1}, atomic_masses)
# Returns 18.015 here on Windows 11, X64
# 18.016 below is probably a typo but it could also be fun with floats :)
# Example: {"H": 2, "O": 1} should return 18.016
```

```
from typing import Dict, List

def calculate_mass_from_list(element_list: List[str], masses: Dict[str, float]):
    """Calculate molecular mass from list format"""
    return sum(masses[element] for element in element_list)
    # Example: ["H", "H", "O"] should return 18.016

calculate_mass_from_list(["H", "H", "O"], atomic_masses)
```

```
from typing import Dict, Tuple

def calculate_mass_from_tuple(formula_tuple: Tuple[Tuple[str, int]], masses: Dict[str, float]):
    """Calculates molecular mass from tuple format;
    Assumes first element in tuple is the element symbol and the second is the number of atoms"""
    return sum(masses[formula[0]]*formula[1] for formula in formula_tuple)
    # Example: (("H", 2), ("O", 1)) should return 18.016

calculate_mass_from_tuple((("H", 2), ("O", 1)), atomic_masses)
```

Which is the “best” option?

Discussion of homework

Exercise 2: pH Buffer System Analyzer

Objective: Work with functions, loops, and data structures to analyze chemical buffer systems.

Problem: Create a system to analyze pH buffer solutions using the Henderson-Hasselbalch equation and manage experimental data.

Your Task: Build functions that calculate buffer properties and store results in appropriate data structures.

```
# Buffer system data (pKa values)
# Note this is a dictionary of dictionaries -- we could instead make a data class!
buffer_systems = {
    "acetic_acid": {"pKa": 4.76, "name": "Acetic Acid/Acetate"},
    "phosphate": {"pKa": 7.20, "name": "Phosphate Buffer"},
    "ammonia": {"pKa": 9.25, "name": "Ammonia/Ammonium"},
    "carbonic": {"pKa": 6.37, "name": "Carbonic Acid/Bicarbonate"}
}
```

- Calculate pH (Henderson-Hasselbalch)
- Find best buffer for target pH
- Calculate pH for concentration ratios
- Rank buffers for suitability (target pH)

How did you solve it?

Buffer System Analyzer

```
def henderson_hasselbalch(pKa, base_conc, acid_conc):  
    """Calculate pH using Henderson-Hasselbalch equation"""  
    # pH = pKa + log([A-]/[HA])  
    # math.log uses the natural log as default but we need log10  
    pH = pKa + math.log10(base_conc/acid_conc)  
    # Alternative: pH = pKa + math.log(base_conc/acid_conc,10)  
    return pH
```

✓ 0.0s

```
pH = henderson_hasselbalch(4.76, 0.1, 0.1)  
print(f"pH with equal concentrations: {pH}") # Should equal pKa (4.76)
```

✓ 0.0s

pH with equal concentrations: 4.76

```
def find_best_buffer(target_pH, buffer_systems):  
    sort_key = lambda buffer: abs(buffer['pKa'] - target_pH)  
    return sorted(buffer_systems.values(), key=sort_key)[0]
```

```
find_best_buffer(7.4, buffer_systems)
```

```
{'pKa': 7.2, 'name': 'Phosphate Buffer'}
```

```
def analyze_buffer_series(buffer_name, pKa, base_concentrations, acid_concentrations):  
    """Calculate pH for multiple concentration ratios"""  
    buffer = buffer_systems[buffer_name]  
  
    results = []  
    for base_conc in base_concentrations:  
        for acid_conc in acid_concentrations:  
            #print(  
                pH = henderson_hasselbalch(buffer['pKa'], base_conc, acid_conc)  
                results.append({'base':base_conc, "pH": pH, "acid": acid_conc})  
            )  
  
    return results
```

```
base_concs = [0.05, 0.1, 0.2]  
acid_concs = [0.1, 0.1, 0.1]  
results = analyze_buffer_series("acetic_acid", 4.76, base_concs, acid_concs)  
for result in results:  
    print(f"[Base]={result['base']}, [Acid]={result['acid']}, pH={result['pH']:.2f}")
```

```
[Base]=0.05, [Acid]=0.1, pH=4.46  
[Base]=0.05, [Acid]=0.1, pH=4.46  
[Base]=0.05, [Acid]=0.1, pH=4.46  
[Base]=0.1, [Acid]=0.1, pH=4.76  
[Base]=0.1, [Acid]=0.1, pH=4.76  
[Base]=0.1, [Acid]=0.1, pH=4.76  
[Base]=0.2, [Acid]=0.1, pH=5.06  
[Base]=0.2, [Acid]=0.1, pH=5.06  
[Base]=0.2, [Acid]=0.1, pH=5.06
```

Buffer System Analyzer

```
def find_best_buffer(target_pH, buffer_systems):
    sort_key = lambda buffer: abs(buffer['pKa'] - target_pH)
    return sorted(buffer_systems.values(), key=sort_key)[0]

find_best_buffer(7.4, buffer_systems)
```

```
{'pKa': 7.2, 'name': 'Phosphate Buffer'}
```

```
def buffer_capacity_rank(buffer_systems, target_pH):
    sort_key = lambda item: abs(item[1]['pKa'] - target_pH)
    return sorted(buffer_systems.items(), key=sort_key)

rankings = buffer_capacity_rank(buffer_systems, 7.0)
print("Buffer rankings for pH 7.0:")
for i, (name, data) in enumerate(rankings):
    print(f"{i+1}. {data['name']} (pKa = {data['pKa']})")
```

```
Buffer rankings for pH 7.0:
1. Phosphate Buffer (pKa = 7.2)
2. Carbonic Acid/Bicarbonate (pKa = 6.37)
3. Acetic Acid/Acetate (pKa = 4.76)
4. Ammonia/Ammonium (pKa = 9.25)
```

Returning one item versus returning a ranked list of items

Feature	<code>find_best_buffer</code>	<code>buffer_capacity_rank</code>
Purpose	To identify and return only the single best buffer, discarding all others.	To provide a ranked list of all available buffers, from most to least effective.
Data Sorted	It sorts the dictionary's <code>values()</code> —a list of just the properties dictionaries.	It sorts the dictionary's <code>items()</code> —a list of (name, properties) tuples.
Output	A single dictionary containing the properties of the best buffer. You lose the buffer's name and the ranking of alternatives.	A complete, sorted list of (name, properties) tuples. You get the buffer's name and its data.

Table made with Gemini

Exercise 3: Reaction Simulator Class

Objective: Create a reaction simulator that models the concentration of the reagents and products as a reaction progresses.

Problem: Build a ChemicalReaction class that simulates simple chemical reactions, tracks reactant consumption, and handles errors when invalid data is provided.

Your Task: Complete the class below:

```
import math
from typing import Dict
from collections import defaultdict
from itertools import chain

class ChemicalReaction:
    def __init__(self, reactants: Dict[str, int], products: Dict[str, int], reaction_rate: int = 1.0):
        """
        Initialize a chemical reaction
        reactants: dict like {"H2": 2, "O2": 1}
        products: dict like {"H2O": 2}
        the terms reagent and reactant are used here interchangeably but reactant is strictly more correct
        The integer values for reactants and products indicate the reaction stoichiometry
        reaction_rate: float (how fast the reaction proceeds, in one reaction given by reactants and products per second)
        """

        # To be initialised
        self.reactants = reactants
        self.products = products
        self.reaction_rate = reaction_rate

        self.reaction_vessel_content: Dict[str, float] = {}

    def add_reactant_amount(self, compound: str, amount: float):
        """Add amount of reactant to the reaction vessel. What happens if the compound added is not part of the reagents or products?"""

    def run_reaction(self, time_seconds: float = 1.0):
        """Run reaction for given time and update reactant amounts. What happens if not enough of one compound is left?"""

    def get_reaction_status(self):
        """Return current content of the reaction vessel as a dictionary."""
```

Activity: GitHub & VCS in VSC 😊

Create a repository via Github, clone and set up uv project (uv init)

Create a python file in the main branch (e.g. use the code on the upper right), then commit

Create a branch in your local repository, create a second file (e.g. a runscript as indicated on the lower right), commit.

Merge the development branch into the master branch (main)

Create a merge conflict: modify the same file and commit on two different branches

Resolve merge conflicts

Push to remote repo

```
def greetings(name: str) → str:
    """Greetings by python"""
    return f"Hello {name}!"

if __name__ == "__main__":
    name = input("What is your name? ")
    print(greetings(name))
```

```
from greetings_script import greetings

print(greetings("World"))
```

Use conventional commits!!

(<https://gist.github.com/qoomon/5dfcdf8eec66a051ecd85625518cfd13>)

Best practices

The repository setup: Define environment, access control (Templates)

Establish workflow alignment: Define coworking rules

Write clear commit messages: Explain the changes in short and concisely. Use standardized nomenclature (e.g. conventionalcommits)

Commit often: Small commits, concise messages.

Test before committing: Use linters (code formatting!) and testing tools (e.g. pytest) and CI workflows to do that automatically

Commit only when ready

Avoid conflicts: Always use up-to-date code, make use of branches!

Use branches: Code branches support multiple versions of software releases and patches. Avoid committing directly to main!

Limit repository access: Only to contributors who need it, control unauthorized access

<https://github.com/resources/articles/software-development/what-is-version-control>

NumPy

Numpy = library for supporting large multidimensional arrays and matrices (including functions to operate on these arrays)

Main benefit:

- Vanilla lists: heterogeneous – each item must contain its own type info additional to the data
- Numpy arrays: homogeneous, each item of the same type – much more efficient for storing and manipulating big data

Usual alias is **np**

```
import numpy as np
np.__version__
```

✓ 0.0s

'2.0.2'

NumPy – creating arrays

Create arrays: `np.array`

- One-dimensional arrays
 - only one index is needed to access entries, e.g. **`onedim[0]`**
- Two-dimensional arrays:
 - 2D arrays have *rows* (horizontal) and *columns* (vertical)
 - two indices are needed: **`array[row, column]`**

dtype specifies the data type in the array!

List of dtypes: <https://numpy.org/doc/stable/user/basics.types.html>

```
onedim = np.array([1, 2, 3, 4, 5])  
print(onedim)
```

✓ 0.0s

```
[1 2 3 4 5]
```

```
twodim = np.array([[1, 2, 3], [4, 5, 6]])  
print(twodim)
```

✓ 0.0s

```
[[1 2 3]  
 [4 5 6]]
```

```
onedim = np.array([1, 2, 3, 4, 5], dtype='float32')  
print(onedim)
```

✓ 0.0s

```
[1. 2. 3. 4. 5.]
```

NumPy – creating arrays

Other options to create arrays:

- **`np.arange(start, end, stepsize)`**
- **`np.zeros((rows, columns))`**
- **`np.ones((rows, columns))`**
- **`np.random.random((rows, columns))`**
- **`np.random.randint(low, high, (rows, columns))`**

NumPy – array attributes

The type of an array is `numpy.ndarray`

```
onedim = np.array([1, 2, 3, 4, 5])  
type(onedim)
```

✓ 0.0s

`numpy.ndarray`

Useful attributes to inspect:

- **`np.ndim`**(array): number of dimensions
- **`np.shape`**(array): size of each dimension
- **`np.size`** (array): total size, i.e. number of all elements

NumPy – 1D slicing

The NumPy slicing syntax follows that of the standard Python list: **x[start:stop:step]**. Note: The stop index will not be included!

Slice	Description
x[:5]	First five elements
x[5:]	All elements after index 5
x[4:7]	Middle subarray
x[::2]	Every other element
x[1::2]	Every other element, starting at index 1
x[::-1]	All elements, reversed
x[5::-1]	Reverses all elements up until index 5 (included)

Slicing can be used to change one or multiple elements (also in 2D arrays):

```
# set the last three values to 333
nums = np.array([1,2,3,4,5, 20, 100, 23, 43, 12])
nums[:-4:-1] = 333
print(nums)
```

✓ 0.0s

```
[ 1  2  3  4  5 20 100 333 333 333]
```

NumPy – 2D slicing

Multiple slices are now separated by commas, first stating the slices of **rows**, then of the **columns**: **Y[start:stop:step, start:stop:step]**

Slice	Description
Y[:2, :3]	First two rows and first three columns
Y[:3, ::2]	First three rows and every other column
Y[::-1, ::-1]	Reverse rows and columns
Y[:, 1]	Second column
Y[3, :]	Fourth row
Y[3]	Same as Y[3, :] , fourth row

NumPy – manipulation of arrays

Copy vs. View!

- With Python **lists**, the slices will be ***copies (duplicated data)***: If we modify the subarray (e.g. change one entry), only the copy gets changed
- With NumPy **arrays**, the slices will be ***direct views (original data)***: If we modify the subarray (view), the original array gets changed too
- When working with large datasets, we don't need to copy any data (slow operation)
- Creating copies: we can use the **`copy()`** method of a slice to create a copy of the specific subarray
- *Note*: The type of a slice is again **`numpy.ndarray`**

NumPy - ufuncs

Looping over normal lists in vanilla python is very slow due to the type checking for each element

Numpy **universal functions (ufuncs)** are faster, vectorized operation for the whole array instead of a classical loop.

Operator	Equivalent ufunc	Description
+	np.add	Addition
-	np.subtract	Subtraction
-	np.negative	Unary negation (e.g., -2)
*	np.multiply	Multiplication
/	np.divide	Division
//	np.floor_divide	Floor division (e.g., 3 // 2 = 1)
**	np.power	Exponentiation (e.g., 3**2 = 8)
%	np.mod	Modulus/remainder (e.g., 9 % 4 = 1)

NumPy – aggregates

Aggregates are useful to compute summary statistics for the data, such as:

- Mean
- Standard deviation
- Median
- Minimum
- Quantiles
- ...

Universal function: compare vanilla operations with vectorized ufuncs (%timeit)

UZH ZI & David Pinezich, ZI APPD course 2024

Function name	Description
np.sum	Calculates sum of elements
np.prod	Calculates product of elements
np.mean	Computes mean of elements
np.std	Calculates standard deviation
np.min	Yields minimum value in elements
np.max	Yields maximum value in elements
np.argmin	Finds index of minimum value
np.argmax	Finds index of maximum value
np.median	Computes median of elements

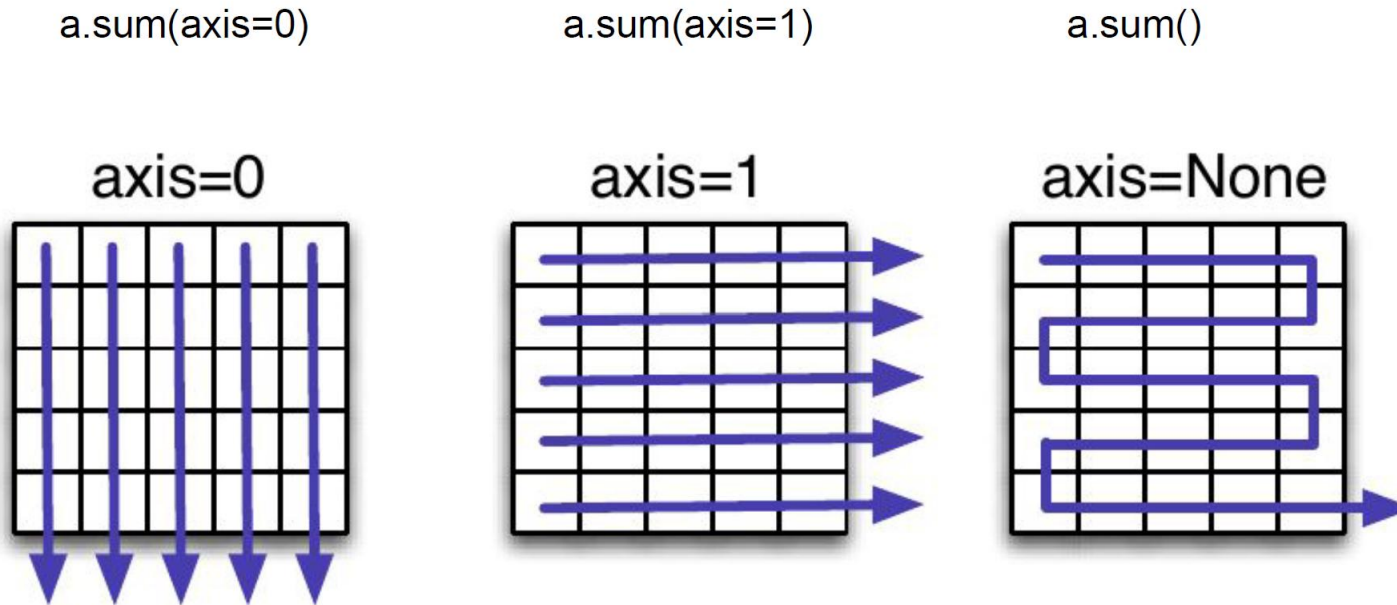
```
%timeit np.sum(random_array) # Numpy ufunc
%timeit sum(random_array) # Python function
✓ 18.4s

1.84 µs ± 19.9 ns per loop (mean ± std. dev. of 7 runs, 1,000,000 loops each)
4.41 µs ± 135 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
```

NumPy – multidimensional aggregates

By default, aggregation functions will be applied to the whole array

For multidimensional arrays, an argument can be provided specifying the axis for the operation:



NumPy – Boolean arrays

NumPy implements comparison operators as element-wise ufuncs, which always results in an **array with a Boolean data type**:

```
np.array([1, 2, 3]) < 2
✓ 0.0s
array([ True, False, False])
```

It is possible to do an element-to-element comparison between two arrays:

False will be interpreted as 0, therefore, the **True** elements can be counted by either the **count_nonzero()** or **sum()** function.

np.all() and **np.any()** will return the Boolean if the condition is met by all or any element, respectively.

Operator	Equivalent ufunc
<code>==</code>	<code>np.equal</code>
<code>!=</code>	<code>np.not_equal</code>
<code><</code>	<code>np.less</code>
<code><=</code>	<code>np.less_equal</code>
<code>></code>	<code>np.greater</code>
<code>>=</code>	<code>np.greater_equal</code>

```
np.array([3,0,8,6,2]) > np.array([2,0,6,6,1])
✓ 0.0s
array([ True, False,  True, False,  True])
```

```
nums = np.array([1,2,3,4,5])
np.count_nonzero(nums < 4)
✓ 0.0s
3
```

NumPy – Boolean arrays

NumPy also implements bitwise logic operators as element-wise ufuncs, which can be used to construct compound conditions (consisting of multiple conditions):

Operator	Equivalent ufunc
&	<code>np.bitwise_and</code>
	<code>np.bitwise_or</code>
^	<code>np.bitwise_xor</code>
~	<code>np.bitwise_not</code>

```
nums = np.array([1,2,3,4,5])
# set all values to 111 for which both conditions are true
nums[(nums < 3) & (nums < 2)] = 111
print(nums)
```

✓ 0.0s

```
[111  2  3  4  5]
```

NumPy – Boolean masks

Boolean arrays from a comparison can be used as **mask to select entries** that meet the same condition:

x			x < 5			x[x < 5]		
3	1	5	True	True	False	3	1	5
10	32	100	False	False	False	10	32	100
-1	3	4	True	True	True	-1	3	4
						▼ (Result)		
						array([3, 1, -1, 3, 4])		

Exercises: Numpy and collaborative coding

Work in pairs:

- 1) Use one of the Github repos you created for the VCS part, copy the two Numpy files (numpy_exercise_A, numpy_exercise_B) from the DSA folder into the repo
- 2) Invite your partner as collaborator to the repo on Github
- 3) You and your partner work on the two notebooks A and B independently, but in different branches.
- 4) Merge branches and compare the results of the exercises

Pandas

Built on NumPy, i.e. also Pandas objects are typed (both keys and values)

More flexible to work with data: attaching labels, working with missing data...

Pandas objects are basically “enhanced” numpy arrays: **Series** (1D), **DataFrames** (2D)

Usually abbreviated and invoked as **pd**:

```
import pandas as pd  
pd.__version__
```

✓ 1.7s

'2.3.3'

Pandas – Series

A Pandas **Series** object is a one-dimensional array of indexed data

- NumPy array: has an *implicitly* defined integer index
- A **Series** object uses by default integer indices
- A **Series** object can have an *explicitly* defined index associated with the values:
- The index labels can be accessed by the **index** attribute:
- If a **Series** is created from a dictionary, the index will be the keys of the dict:

```
data1 = pd.Series([1, 2, 3, 4, 5])  
✓ 0.0s
```

```
data2 = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])  
✓ 0.0s
```

```
d2index = data2.index  
print(d2index)  
✓ 0.0s  
Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
```

```
data3 = pd.Series({'b': 1, 'e': 2, 'c': 3, 'd': 4, 'a': 5})  
print(data3.index)  
✓ 0.0s  
Index(['b', 'e', 'c', 'd', 'a'], dtype='object')
```

Pandas - DataFrame

- A **DataFrame** object is an analog of a two-dimensional array both with flexible row indices and flexible column names
- Both the rows and columns have a generalized index for accessing the data
- The row indices can be accessed by using the **index** attribute
- The column indices can be accessed by using the **columns** attribute
- **DataFrame** = sequence of aligned **Series** objects (column = **Series**)

columns (axis 1)

	foo	bar
a	0.652252	0.324959
b	0.731989	0.088452
c	0.557951	0.729868

index (axis 0)

Pandas – Constructing a DataFrame

- From a single Series object:

```
population = pd.Series([1.4, 1.3, 0.3, 0.2], index=['China', 'India', 'USA', 'Indonesia'])
pd.DataFrame(population, columns=['Population (in billions)'])
```

✓ 0.0s

- From a list of dictionaries:

```
pd.DataFrame([{'a': 1, 'b': 2}, {'b': 3, 'c': 4}])
```

✓ 0.0s

- From a dictionary of Series objects:

```
area = pd.Series({'California': 423967, 'Texas': 695662,
                  'New York': 141297, 'Florida': 170312,
                  'Illinois': 149995})
```

```
population = pd.Series({'California': 38332521, 'Texas': 26448193,
                        'New York': 19651127, 'Florida': 19552860,
                        'Illinois': 12882135})
```

```
data = pd.DataFrame({'area': area, 'population': population})
```

- From a two-dimensional NumPy array:

```
pd.DataFrame(np.random.rand(3, 2),
             columns=['foo', 'bar'],
             index=['a', 'b', 'c'])
```

✓ 0.0s

	foo	bar
a	0.652252	0.324959
b	0.731989	0.088452
c	0.557951	0.729868

Pandas – Data Selection: Series

Series can be treated as **dictionaries** (accessing and modifying, looping through, ...)

- Selecting elements by e.g. **data["a"]** or **data[0]** still works but is deprecated

or:

Series can be treated as **one-dimensional array**:

- Select elements by the implicit integer indexer **iloc**, e.g. **data.iloc[0]**
- Select elements by the explicit indexer **loc**, e.g. **data.loc['a']**
- Select slices (by using an implicit integer index or an explicit index)
- *Important:* Slicing with an explicit index (e.g., **data['a':'c']**) will *include* the final index in the slice, while slicing with an implicit index (e.g., **data[0:3]**) will *exclude* the final index from the slice
- Use masking operations, e.g. **data[data < 3]**

Pandas – Data Selection: DataFrame

DataFrame can be treated as a **dictionary of related Series** objects:

- Select Series by the column name, e.g. **df['area']**
- Modify the **DataFrame** object with familiar syntax, e.g. **df['c3'] = df['c2']/ df['c1']**

or:

DataFrame can be treated as **two-dimensional array**:

- Access the underlying NumPy data array by using the **values** attribute, e.g. **df.values[0]** will select the first row
- Use the **iloc** indexer to index, slice, and modify the data by using the *implicit* integer index
- Use the **loc** indexer to index, slice, and modify the data by using the *explicit* index

Pandas – Adding and removing columns to a DataFrame

- Some methods to add columns:
 - Columns can be added by directly declaring a list as column: **df["new_column"] = values**
 - This is similar to the **assign()** method: **df.assign(name = [values])**
 - To add new columns at arbitrary positions, not just at the end, use the **insert()** method: **df.insert(loc, column, value, allow_duplicates)**
 - Adding multiple columns can be done by merging with another DataFrame using **pd.concat([df1, df2], axis=1)**
- Some methods to remove columns:
 - By name: **df.drop(["col names"], axis = 1, inplace=True)**, where **inplace** specifies if a copy is created or the df is changed directly
 - By index: **df.drop(df.columns[[0, 1, 3]], axis=1, inplace=True)**
 - (Slicing)

Pandas – Adding and removing rows to a DataFrame

- Some methods to add rows:
 - Directly defining a new row with the **.loc** indexer: **df.loc[len(df)] = [values]** (avoid when working with explicit indices!)
 - Using **_append()** to add a new row from a dictionary (matching keys!): **df = df._append({'col1': 'a', 'col2': 1234, 'col3': 23.45}, ignore_index=True)**
 - The **_append()** method can also be used to add a **Series**
 - Use **concat([df1, df2])** to merge with another DataFrame to add multiple lines
- Rows can be removed using the **drop()** method:
 - By label (explicit index): **df.drop("Entry1", inplace=True)**
 - By index: **df.drop(df.index[0], inplace = True)**

Pandas ufuncs

Pandas is designed to work with Numpy, hence any NumPy ufunc will work on Pandas **Series** and **DataFrame** objects

- *Index preservation*: Indices are preserved when a new Pandas object will come out after applying ufuncs
- Missing data is marked with **NaN** ("Not a Number", can be checked using **isnull()** or **isna()** function)
- We can specify on how to fill value for any elements that might be missing by using the option **fill_value**: **A.add(B, fill_value=0)**
- We can also use the **dropna()** method to drop entries with missing values
- *Note*: Any of the ufuncs discussed for NumPy can be used in a similar manner with Pandas objects

Pandas - Aggregation

- As with one-dimensional NumPy array, for a Pandas **Series** aggregates return a single value
- For a **DataFrame**, the aggregates return by default results within each column
- Pandas **Series** and **DataFrames** include all common NumPy aggregates
- In addition, there is a convenient method **describe()** yielding several common aggregates for each column and returns the result
- The **aggregate()** method can be applied for multiple operations, specifying operation and column explicitly (e.g. {"A": "sum", "B": "mean"})

- The **filter()** method allows you to drop data based on group properties *Note: **filter()** takes as an argument a *function* that returns a Boolean value specifying whether the group passes the filtering*
- While aggregation must return a reduced version of the data, **transform()** can return some transformed version of the full data to recombine (meaning that we still have the same number of entries before and after the transformation)
- The **apply()** method applies an arbitrary function to the group results. The function should take a **DataFrame**, and return either a Pandas object or a scalar

Exercises: Pandas

In pairs of two work through the Pandas puzzles:

- Arrangement for the exercises 5-17: Person A works on the uneven numbers, Person B on the even numbers
- Define the coworking rules (branches, file names, ...) for your repository prior to starting!

Pandas – Reading and Writing

Read from file:

- Pandas provides the **pd.read_csv()** function to load data from a CSV file from a specified path (locally or URL)
- An optional argument **error_bad_lines** can be set to **False** so that bad lines in the file are ignored when read
- More optional arguments in the documentation: https://pandas.pydata.org/pandas-docs/stable/generated/pandas.read_csv.html

Write to file:

- The **to_csv()** method exports a **DataFrame** to a CSV file
- More details and options: https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.to_csv.html

Of course, CSV files are not the only file that Pandas can process. More methods to import and export data in the documentation: <https://pandas.pydata.org/pandas-docs/stable/api.html#id12>